체크박스(☐)를 클릭하면 체크되고, Edge·크롬·Adobe Reader 에서 저장(Ctrl+S)하면 남습니다.

조각김치 이물검출 라벨링 프로젝트 - 팀 실행 가이드

2026-10-03 · 작성 Sung Hyung Lee (PM) · 6인 팀 공용

한눈에 보기

목표: 10/21까지 900장의 기존 YOLO 라벨을 전수 검수·보정해 FINAL 데이터를 확정하고, 라벨링 프로그램과 함께 발표합니다. 사각형을 많이 그리는 프로젝트가 아니라, 이미 있는 라벨을 믿을 수 있는 정답으로 만드는 프로젝트입니다.

항목	핵심
데이터	4K JPG 900장 + YOLO TXT 900개 (Dataset1/train, Dataset2/train·validation), 100% 가상 생성, NDA
작업 방식	기존 TXT 자동 Load → 사람이 이미지와 비교 → 맞으면 PASS / 고치면 EDITED / 애매하면 REVIEW
Class	0~6 그대로 유지. **4번(고무장갑)은 미사용** → 발견 시 지우지 말고 REVIEW
폴더	RAW(원본, 수정 금지) → WORK(작업 저장) → FINAL(QA 통과분만)
장면 유형	kimchi_with_target / normal_kimchi / object_only / other_review (Class 아님, Manifest에만 기록)
검수	수정·신규·REVIEW·Class 4·빈 TXT는 100% 교차검수, PASS는 25% 표본검수
프로그램	`kimchi_labeler` 최종본 (WSL + .venv, `bash setup_wsl.sh` → `bash run.sh`)
완료 조건	프로그램 + 검수 데이터 + QA 기록 + 문서 + 교과 8 Handoff
일정	프로젝트 세션 11회 (10/6~10/21, 매일 14:00~18:00, 총 44시간). Freeze 10/7, 본 검수 10/8~10/14, FINAL 10/19
인원	6명, 1인당 150장 담당 + 다른 1명의 작업 검수

최종 품질 목표 (모두 0이어야 FINAL 생성 가능)

지표	목표
☐ Image-Label Pair Error	0
☐ TXT Parsing Error	0
☐ Unknown Class Error	0
☐ BBox Critical Range Error	0
☐ 미처리 REVIEW / PENDING	0
☐ RAW 변경	0 (해시 검증)

이 숫자는 모델 성능이 아닙니다. 교과 8에서 바로 학습할 수 있을 만큼 데이터가 준비되었다는 뜻입니다.

프로젝트 핵심 이해하기

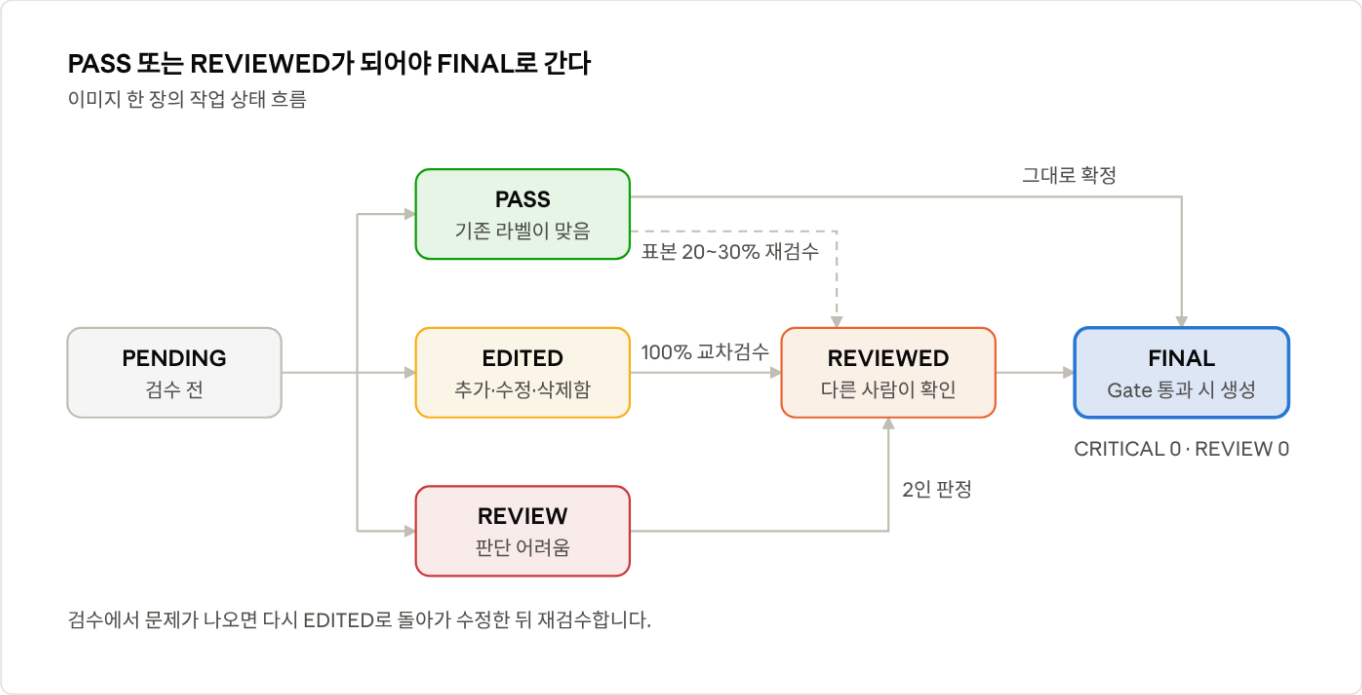
한 문장으로: AI가 배울 '정답지'를 검수해서, 틀린 답을 고치고 그 과정을 기록으로 남기는 프로젝트입니다. 교과 8에서 모델은 이 정답지를 보고 공부하므로, 정답지가 틀리면 모델도 틀리게 배웁니다.

비유하면 이렇습니다. 회사가 답이 적힌 문제집 900장(사진 + TXT)을 줬고, 우리는 채점자입니다. 문제집 원본(RAW)에는 펜을 대지 않고, 복사본(WORK)에만 빨간 펜을 씁니다. 다른 채점자가 한 번 더 보고(교차검수), 통과한 것만 최종본(FINAL)으로 묶습니다.

데이터가 지나가는 길

1. RAW: 회사가 준 JPG와 TXT. 읽기만 하고 절대 고치지 않습니다. 처음에 해시를 기록해 안 바뀜을 증명합니다.
2. 프로그램: JPG와 같은 이름의 TXT를 자동으로 읽어 BBox를 그립니다.
3. 사람: 사진과 비교해 맞으면 PASS, 고치면 EDITED, 애매하면 REVIEW.
4. WORK: 저장한 TXT와 Manifest(누가, 언제, 무엇을, 왜)가 쌓입니다.
5. 검사: 자동 Validation(기계가 형식 오류 찾기) + 교차검수(사람이 내용 확인).
6. FINAL: 조건을 모두 통과해야만 생성되는 최종 데이터. 교과 8로 넘어갑니다.

작업 상태는 이렇게 바뀝니다



프로그램은 라벨을 바꾼 뒤 PASS를 누르면 자동으로 EDITED로 저장하고, Class 4가 남아 있으면 REVIEW로 저장합니다. FINAL은 사람이 고르는 상태가 아니라 `07_build_final.py`가 조건을 확인한 뒤 붙입니다.

YOLO 라벨 한 줄 읽는 법

```
3 0.512000 0.430000 0.060000 0.045000
```

값	뜻	이 예시에서
3	Class 번호	벌레류
0.512	BBox 중심 X (이미지 너비 대비)	가로 51.2% 지점

0.430	BBox 중심 Y (이미지 높이 대비)	세로 43% 지점
0.060	BBox 너비	이미지 너비의 6%
0.045	BBox 높이	이미지 높이의 4.5%

모든 값이 0~1 비율이라 이미지 크기가 바뀌어도 같은 위치를 가리킵니다. 프로그램은 이것을 원본 픽셀 좌표로 바꿔 그리고, 저장할 때 다시 비율로 돌립니다. 빈 TXT는 '이물이 없다'는 뜻일 수도, '라벨이 빠졌다'는 뜻일 수도 있어서 사람이 사진을 보고 구분합니다.

Class와 장면 유형은 다릅니다

구분	무엇을 나타내나	어디에 기록하나	예
Class (0~6)	BBox 안 **물체의 종류**	YOLO TXT	2 나뭇가지류
장면 유형 (scene_type)	**사진 전체의 형태**	Manifest	김치 + 대상 객체 / 정상 김치 / 대상 객체 단독

그래서 '정상 김치'를 Class 7로 만들지 않습니다. Class 4(고무장갑)는 번호는 있지만 쓰지 않는 Class라서, 발견하면 지우지 않고 REVIEW로 보냅니다.

교과 8에서 이어지는 것

- FINAL 데이터로 YOLO11 모델을 학습하고, Precision·Recall·mAP로 평가합니다.
- Manifest의 출처(Dataset 1/2, train/validation)와 장면 유형을 보고 학습·검증 데이터를 나눕니다. 그래서 교과 7에서 이 정보를 섞거나 지우면 안 됩니다.
- 교과 7의 보정 통계(추가·삭제·Class 변경 수)는 '데이터가 얼마나 믿을 만한가'의 근거가 됩니다.

꼭 지켜야 할 규칙

아래 신호가 하나라도 보이면 그 자리에서 대량 작업을 멈추고 PM과 QA에게 알립니다. 나중에 고치면 900장을 다시 봐야 할 수 있습니다.

중단 신호	확인 방법
저장이나 Zoom 후 BBox 위치가 바뀜	`09_golden_test.py` 재실행
재실행하면 라벨이 사라짐	WORK 경로와 저장 성공 메시지 확인
Class ID가 저장 중 바뀜	원본 TXT와 비교
다른 이미지의 TXT를 덮어씀	Manifest의 담당자와 경로 확인
RAW가 수정됨	`00_raw_snapshot.py --verify`
다음 이미지에 이전 BBox가 남음	이미지 전환 테스트 20장

대응 순서: 작업 중지 → 문제 재현 → 영향 범위 확인 → 수정 → Golden Sample → Pilot 재시험 → 재개

데이터 의미를 바꾸지 않습니다

- Class 번호를 0~5로 다시 매기지 않습니다. Class 4는 삭제하지 않고 REVIEW로 보냅니다.
- 정상 김치 이미지와 객체만 있는 이미지를 삭제하지 않습니다.
- 기존 validation 폴더를 train에 섞지 않습니다. split은 교과 8에서 결정합니다.
- 빈 TXT를 무조건 오류로 보지 않습니다. 정상 이미지인지, 라벨이 빠진 것인지 사람이 확인합니다.

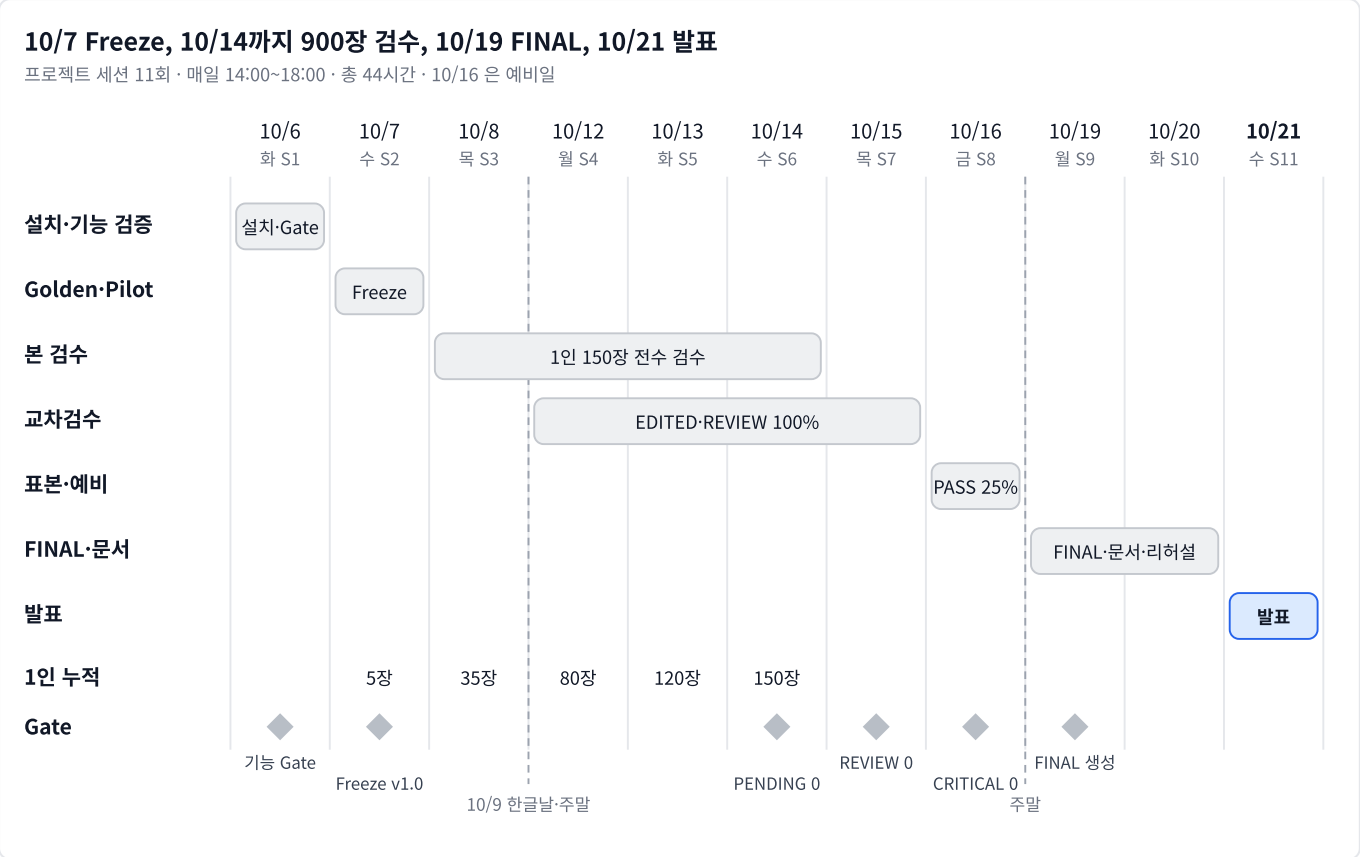
- 애매하면 추측하지 말고 REVIEW로 보내고, 2명이 확인한 뒤 Note에 근거를 남깁니다.

협업과 보안

- 같은 이미지는 담당자만 수정합니다. 프로그램의 `내 담당` 보기로만 작업합니다.
- **RAW 폴더는 6명 모두 같은 경로**(`~/kimchi_labeler/data/raw`)에 둡니다. 그러면 WORK 는 모두 `data/work`, FINAL 은 `data/final` 이 되어 병합본을 그대로 바꿔 끼울 수 있습니다.
- 매일 17:30에 각자의 WORK 폴더를 공유폴더로 모으고, `04_merge.py`로 합칩니다.
- Git에는 코드와 문서만 올립니다. JPG, TXT, WORK, FINAL, settings.json 은 올리지 않습니다 (.gitignore 적용됨).
- **10/7 Feature Freeze** 이후에는 버그 수정 외에 기능을 추가하지 않습니다. 개선 실습은 별도 브랜치에서만 합니다.

일정

프로젝트 시간은 10/6~10/21 평일 오후 14:00~18:00, 세션 11회(44시간)입니다. 10/5(개천절 대체공휴일)와 10/9(한글날), 주말은 뺐습니다. 매일 오전 9~13시는 수업, 13~14시는 점심입니다.



프로그램이 완성되어 있으므로 **앞 2회(10/6~10/7)에 설치·검증·Pilot 을 끝내고 10/7 에 Freeze** 합니다. 10/8~10/14 네 번에 본 검수 900장(1인 150장, 하루 30~45장)을 끝내고, 10/15 교차검수 100%, 10/16 PASS 표본검수와 **예비일**, 10/19 FINAL, 10/20 문서·리허설 순서입니다.

세션	날짜	목표	1인 누적 / 전체
S1	10/6 (화)	설치, 실제 데이터 연결, 기능 Gate	-
S2	10/7 (수)	Golden, 배분, Pilot 30장 → **Feature Freeze v1.0**	5장 / 30
S3	10/8 (목)	본 검수 1	35장 / 210 (23%)
S4	10/12 (월)	본 검수 2 + 교차검수 시작	80장 / 480 (53%)

S5	10/13 (화)	본 검수 3 + 교차검수	120장 / 720 (80%)
S6	10/14 (수)	본 검수 완료 (**PENDING 0**)	150장 / 900 (100%)
S7	10/15 (목)	교차검수 100% (**EDITED·REVIEW 0**)	-
S8	10/16 (금)	PASS 표본검수 25%, **CRITICAL 0**, 예비일	-
S9	10/19 (월)	**FINAL 생성**, Acceptance Test, 문서 생성	-
S10	10/20 (화)	문서 확정, 발표 리허설 2회	-
S11	10/21 (수)	발표	-

매 세션 시간 배분

시간	할 일
14:00~14:10	시작 회의: 오늘 목표와 Gate 공유, 전날 병합본 배포 (PM)
14:10~17:30	세션별 작업 (아래 체크리스트). S4 부터는 첫 40분을 전날 분량 교차검수에 씁니다.
17:30~18:00	마감 루틴: 병합, Validation, QA Summary, 다음 날 목표 공지

세션별 완료 목표와 체크리스트

매 세션은 14:00 시작 회의 10분(오늘 Gate 공유), 17:30 마감 루틴 30분으로 나눕니다. **체크박스가 모두 채워져야 다음 세션 목표로 넘어갑니다.** 괄호는 책임 역할입니다.

S1 · 10/6(화) 설치 · 실제 데이터 연결 · 기능 Gate

완성 기준: 6명 모두 최종 프로그램으로 실제 900장을 열 수 있고, RAW 기준선이 기록되어 있고, 기능 Gate(탐색·편집·Zoom·저장)를 실제 데이터로 통과합니다.

☐ zip 받아 `bash setup_wsl.sh` → "모든 테스트 통과 (4개)" (전원)

☐ RAW 를 `~/kimchi_data/raw` 에 두고 `bash run.sh --raw ~/kimchi_data/raw` , 작업자 이름 입력 (전원)

☐ RAW 해시 기록 `00_raw_snapshot.py` (PM)

☐ Manifest 생성 + Data Inventory 기록 `01_build_manifest.py` (문서)

☐ RAW 사전 검사 `02_validate.py --target raw` , 오류 유형별 개수 공유 (QA)

☐ 20장 이상 연속 이동 시 멈춤·BBox 잔상 없음, 메뉴 6개 빠르게 눌러도 잔상 없음 (GUI)

☐ TXT 줄 수 = 화면 BBox 수, 여러 장에서 확인 (저장)

☐ 새 BBox, Class 지정·변경, 삭제·재작성, 이동·크기 조절, 좌표 입력, 다중 BBox (좌표)

☐ 실수 드래그와 이미지 밖 BBox가 만들어지지 않음, 확대 상태에서 그린 BBox가 Fit 후에도 같은 위치 (좌표)

☐ 저장 → 다음 → 이전 시 수정 유지, 재실행 후 복원, 저장 안 하고 이동·닫기 시 경고 (저장·GUI)

☐ Git 저장소 만들고 데이터·settings.json 이 올라가지 않는지 확인, 이 PDF 배포 (PM)

S2 · 10/7(수) Golden · 배분 · Pilot → Feature Freeze

완성 기준: Golden Sample 실패 0장, 900장이 6명에게 배분, Pilot 30장 Critical 0건, 병합 리허설 성공, 프로그램 v1.0 고정.

- ☐ Golden Sample 10~20장 사람이 먼저 확인 → `09_golden_test.py --list golden.txt` 실패 0장 (QA)
- ☐ scene_type 4가지 판단 기준 합의, 예시 이미지 각 2장 (PM·전원)
- ☐ `03_assign.py --workers A B C D E F --pilot 30` → 배분된 WORK 를 전원 `data/work_raw` 로 배포 (PM)
- ☐ Pilot 30장 처리 (`Pilot` 보기, 1인 5장) (전원)
- ☐ Pilot 교차검수 + Validation 으로 실제 오류를 찾는지 확인 (QA)
- ☐ 2명이 3장씩 저장 → `04_merge.py` 병합 리허설, 충돌 0건 (문서)
- ☐ 상태 규칙 시험: 수정 후 PASS → EDITED, Class 4 → REVIEW (저장)
- ☐ 1장 평균 처리시간 측정 후 남은 일정 재계산 (PM)
- ☐ RAW 변경 없음 `00_raw_snapshot.py --verify` (QA)
- ☐ **Feature Freeze** 선언, Git 태그 `v1.0` (PM)

S3 · 10/8(목) 본 검수 1

완성 기준: 1인 누적 35장 (전체 210/900, 23%).

- ☐ 역할별 코드 설명 5분씩 (팀 코드 리뷰, 14:10~14:40) (전원)
- ☐ `내 담당-미완료` 보기로 본 검수 30장 (전원)
- ☐ 검수 순서 지키기: 전체 → 네 구역 → 중심 → 기존 BBox 주변 확대 (전원)
- ☐ EDITED·REVIEW 는 Note 에 이유 기록 (전원)
- ☐ 17:30 병합 → Validation → QA Summary (문서·QA)

S4 · 10/12(월) 본 검수 2 + 교차검수 시작

완성 기준: 1인 누적 80장 (480/900, 53%), 10/8 분량 EDITED·REVIEW 교차검수 완료.

- ☐ 14:10~14:50 `내 검수 대상` 보기로 10/8 분량 교차검수 (전원)
- ☐ 본 검수 45장 (전원)
- ☐ REVIEW 목록 2인 판정, 판단 근거 기록 (PM·QA)
- ☐ 담당자별 진행률 확인, 뒤처진 사람은 다음 날 재배분 검토 (PM)
- ☐ 병합 충돌 보고서 `reports/merge_conflicts.csv` 0건 (문서)

S5 · 10/13(화) 본 검수 3

완성 기준: 1인 누적 120장 (720/900, 80%), 전날 분량 교차검수 완료.

- ☐ 전날 분량 교차검수 (전원)
- ☐ 본 검수 40장 (전원)
- ☐ CRITICAL Validation 감소 추이 확인 (QA)
- ☐ 발표 자료 뼈대 작성 (문서·PM)

S6 · 10/14(수) 본 검수 완료

완성 기준: PENDING 0장, 1인 150장 완료, 교차검수 70% 이상.

- ☐ 남은 본 검수 30장 완료 (전원)
- ☐ 전날 분량 교차검수 (전원)
- ☐ 진행현황에서 PENDING 0 확인 (PM)
- ☐ RAW 변경 없음 `00_raw_snapshot.py --verify` (QA)

S7 · 10/15(목) 교차검수 100%

완성 기준: EDITED 0장, REVIEW 0장 (모두 REVIEWED 또는 판정 완료).

- ☐ 남은 EDITED·REVIEW 교차검수 (전원)
- ☐ REVIEW 2인 판정 마무리, Note 근거 확인 (PM·QA)
- ☐ Validation CRITICAL 이미지를 담당자에게 전달해 수정 (QA)
- ☐ 데이터 현황 통계 정리 (Dataset/split, 장면 유형, 보정 내역) (문서)

S8 · 10/16(금) PASS 표본검수 + 예비일

완성 기준: PASS 표본 25% 검수 완료, 전체 Validation CRITICAL 0건. 밀린 분량이 있으면 이 날 끝냅니다.

- ☐ `06_pass_sample.py --ratio 0.25` 실행 후 배포 (QA)
- ☐ PASS 표본검수: 이상 없으면 REVIEWED, 문제가 있으면 수정 후 EDITED (전원)
- ☐ 표본에서 오류가 많이 나온 작업자는 PASS 범위를 더 넓게 재검수 (PM)
- ☐ 전체 Validation CRITICAL 0건 (QA)
- ☐ 밀린 검수·교차검수 처리 (예비 시간) (전원)
- ☐ 발표 자료 초안 완성 (문서·PM)

S9 · 10/19(월) FINAL · Acceptance Test

완성 기준: FINAL 폴더 생성, Acceptance Test 통과, 산출물 문서 생성.

- ☐ 위험 이미지 위주 Human Final QA: 새 BBox, Class 변경, 작은 BBox, 희소 Class (QA)
- ☐ `07_build_final.py` 성공 (PM)
- ☐ 만든 사람이 아닌 팀원이 README만 보고 Acceptance Test 16개 항목 통과 (QA)
- ☐ `05_qa_summary.py`, `08_handoff.py` 로 QA Summary와 Handoff 생성 후 다듬기 (문서)
- ☐ Test Report (Golden·Pilot·Acceptance) 작성 (QA·문서)
- ☐ 시연 2분 화면 녹화본 만들기 (GUI)

S10 · 10/20(화) 문서 확정 · 리허설

완성 기준: 11개 산출물 완성, 발표 리허설 2회.

- ☐ 11개 산출물 최종 점검 (아래 표) (PM)
- ☐ 발표 자료 완성 (문서·PM)
- ☐ 발표 리허설 2회, 시간 측정 (전원)
- ☐ STUDY_GUIDE 13장 예상 질문 답변 연습 (전원)
- ☐ 시연 PC에서 프로그램과 FINAL 데이터 동작 확인 (GUI)

S11 · 10/21(수) 발표

완성 기준: 시연과 최종 결과 발표 완료.

☐ 시연 PC 최종 확인 (GUI)

☐ 최종 리허설, 발표자별 파트 점검 (PM)

☐ 발표 (전원)

☐ Git 최종 커밋 태그 `final`, 산출물 제출 (PM)

역할별 코딩 담당과 완성 기준

프로그램 완성본(`kimchi_labeler`)은 이미 동작합니다. 각자의 일은 **담당 파일을 이해하고, 실제 데이터로 검증하고, 발표에서 설명하는 것**입니다. 코드를 한 파일씩 해석한 공부 자료는 `docs/STUDY_GUIDE.md`, 역할별 요약은 `docs/CODE_GUIDE.md`에 있습니다.

역할	담당 파일	10/7(Freeze)까지 완성 기준	발표에서 설명할 것
PM / Integrator	`config.py`, `main.py`, `setup_wsl.sh`, `run.sh`, `03_assign.py`, `07_build_final.py`	전원 실행 환경 통일, Git 운영, 배분과 FINAL 조건 이해	기준선, 일정, FINAL Gate를 코드로 만든 이유
GUI / Image	`ui/app.py`, `panels.py`, `menubar.py`, `dialogs.py`, `style.py`	이미지 목록·모드 전환·미저장 경고·메뉴 동작 확인, 4K 표시 속도 점검	화면 구성, 피라미드·보이는 영역만 그리기, 메뉴를 직접 만든 이유 (WSLg 잔상)
BBox / Coordinate	`bbox/view.py`, `yolo/yolo_io.py` 변환 함수, `bbox/canvas_editor.py` 마우스 부분	Zoom/Pan·이동·크기 조절 후 좌표 일치, Golden 오차 0	원본 좌표로만 저장하는 이유, 줌 공식, drag 상태 기계
Label / Storage	`yolo/yolo_io.py`, `bbox/session.py`, `yolo/dataset.py`	Save/Reload, 저장 유실 방지, RAW 무수정 확인	안전저장, PASS→EDITED 자동 규칙
QA / Validation	`validation/validator.py`, 02, 06, 09, 00 스크립트	RAW 오류 목록, Golden Sample 목록과 통과	오류 코드 표, QA 결과 숫자
Data / Document	`manifest.py`, `stats.py`, 01, 04, 05, 08 스크립트	Manifest·병합 리허설, 문서 템플릿	Manifest 출처 보존, 보정 내역 통계

모든 역할 공통

- 코드를 고칠 때는 PLAN → CODE → RUN → CHECK → RECORD 순서를 따릅니다. 고친 뒤에는 `python tests/test_core.py` 와 `python scripts/09_golden_test.py` 를 반드시 다시 돌립니다.
- 자기 파일에서 필수 함수 3개를 골라, 다른 팀원에게 말로 설명할 수 있어야 합니다. 발표 예상 질문은 STUDY_GUIDE 13장에 있습니다.
- 각자 150장 검수는 역할과 관계없이 모두 합니다. 모든 팀원이 라벨 확인, 새 BBox, 수정·삭제, 다른 사람 작업 검수를 직접 경험해야 합니다.
- Feature Freeze(10/7) 이후의 개선 실습은 별도 브랜치에서만 합니다. STUDY_GUIDE 12장 연습 문제를 참고하세요.

프로그램 사용법 (최종본)

팀 공용 환경은 **Windows 11 + WSL2(Ubuntu 22.04 이상) + 가상환경(.venv)** 입니다. 아래 명령은 모두 **WSL 터미널**에서 줄이 나뉜 그대로 붙여 넣습니다. 배포 파일은 `kimchi_labeler.zip` 하나입니다.

처음 설치 (PC마다 한 번)

```
# 1) kimchi_labeler.zip 을 Windows '다운로드' 폴더에 받은 뒤, WSL 홈(~)에 풀기
ZIP=$(ls -t /mnt/c/Users/*/Downloads/kimchi_labeler*.zip | head -1)
echo "$ZIP"
python3 -m zipfile -e "$ZIP" ~/

# 2) 설치: 패키지 · 한글 글꼴 · .venv · 연습 데이터 · 자동 테스트를 한 번에
cd ~/kimchi_labeler
bash setup_wsl.sh
```

- WSL 비밀번호를 물어보면 입력합니다 (글자가 안 보이는 것이 정상).
- 마지막에 **"모든 테스트 통과 (4개)"** 와 **"설치 완료."** 가 나오면 준비 끝입니다.
- 창 제목줄의 한글이 네모로 보이면 **Windows PowerShell** 에서 `wsl --shutdown` 을 한 번 실행하고 WSL 을 다시 엽니다.

RAW 데이터 위치와 첫 실행

```
# 실제 데이터: 6명 모두 같은 경로에 둡니다 (이 안에 이물검출_학습데이터1, 이물검출_학습데이터2)
mkdir -p ~/kimchi_labeler/data/raw
# ... 받은 RAW 데이터를 ~/kimchi_labeler/data/raw 안에 복사 ...
cd ~/kimchi_labeler
bash run.sh --raw data/raw              # 처음 한 번 (폴더는 기억됨)

# 연습할 때: 가짜 데이터 (setup_wsl.sh 가 이미 만들어 둠)
bash run.sh --raw data/dummy_raw
```

- 처음 실행하면 **작업자 이름**을 물어봅니다. 한 번 입력하면 기억합니다.
- RAW 가 `data/raw` 면 WORK 는 자동으로 `data/work`, FINAL 은 `data/final` 이 됩니다 (예시 산출물 구조와 같음). 연습 데이터는 `data/work_dummy_raw`.
- `data/` 는 `.gitignore` 에 들어 있어 Git 에 올라가지 않습니다. 프로그램 폴더 자체를 RAW 로 고르면 "[중단] RAW 와 WORK 는 서로 다른 독립 폴더여야 합니다" 가 뜹니다.

매일 실행

```
cd ~/kimchi_labeler
bash run.sh
```

- 끝낼 때는 창의 X, 파일 → 종료, `Ctrl+Q`, 터미널 `Ctrl+C` 모두 저장 확인 후 종료됩니다.
- RAW 폴더를 바꿀 때는 메뉴 **파일 → RAW 폴더 열기** (`Ctrl+O`).

스크립트(00~10) 실행

```
cd ~/kimchi_labeler
source .venv/bin/activate             # 터미널을 새로 열 때마다 한 번
python scripts/02_validate.py          # 예: WORK 기준 Validation
```

스크립트는 `settings.json` 에 기억된 RAW/WORK 를 자동으로 씁니다. 다른 폴더를 볼 때만 `--raw` , `--work` 를 붙입니다.

새 버전으로 업데이트 (내 작업 데이터는 그대로)

```
ZIP=$(ls -t /mnt/c/Users/*/Downloads/kimchi_labeler*.zip | head -1)
echo "$ZIP"
rm -rf ~/kimchi_final && python3 -m zipfile -e "$ZIP" ~/kimchi_final
cp ~/kimchi_final/kimchi_labeler/update.sh ~/kimchi_labeler/
cd ~/kimchi_labeler && bash update.sh "$ZIP"
rm -rf ~/kimchi_final
bash run.sh
```

`update.sh` 는 코드(`src` · `scripts` · `tests` · `configs` 등)만 바꾸고 `data/` , `settings.json` , 팀이 쓴 문서는 그대로 둡니다.

화면 구성

위치	내용
메뉴	파일 · 보기 · 도구 · 검수 · 통계(진행현황, Class 통계, QA Summary) · 도움말. `Alt+F/V/T/Q/S/H` 로도 열립니다.
왼쪽	이미지 목록 (◯대기 ✓PASS ✎EDITED !REVIEW ✔검수완료), 클릭하면 이동, 보기 필터
가운데	파일명 (현재/전체), 출처·크기, 이미지와 BBox
오른쪽 1열	Class 선택, 선택된 BBox 정보 (X·Y·너비·높이 직접 입력 가능), BBox 목록
오른쪽 2열	작업 상태, Scene Type, 작업자·검수자, 파일 정보, 이슈·메모
아래	빨간 경고 줄 · 보기 도구 · 라벨 도구 (누른 버튼이 파란색) · 이전/다음, 초록 저장, 파랑 저장 후 다음

단축키

동작	키 / 마우스
새 BBox	`W` 모드에서 드래그 (클릭하면 BBox 선택)
BBox 이동·크기 조절	`V` 모드에서 BBox 드래그, 흰 핸들 8개로 크기 조절
Class 선택·변경	`0` ~ `6` (숫자 키패드도 가능, 4는 사용 불가)
삭제 / Undo / 선택 해제	`Del` / `Ctrl+Z` / `Esc`
이전 / 다음 / 다음 미완료	`A` ← / `D` → / `N`
저장 후 다음 / 저장	`Space` / `Ctrl+S`
Zoom / Pan / Fit / 라벨 숨김	휠 또는 `+` `-` / 오른쪽 드래그 / `F` / `H`
상태	`P` (PASS), `E` (EDITED), `R` (REVIEW)
폴더 열기 / 종료 / 도움말 / 메뉴	`Ctrl+O` / `Ctrl+Q` / `F1` / `F10`

한 장 검수 순서

1. 왼쪽 필터를 `내 담당-미완료` 로 둡니다.

2. 전체 → 왼쪽 위 → 오른쪽 위 → 왼쪽 아래 → 오른쪽 아래 → 중심 순서로 확대해 봅니다. 기존 BBox가 물체를 가리면 `H` 로 잠시 숨깁니다.
3. 빠진 이물은 추가, 틀린 Class는 변경, 위치가 어긋난 BBox는 `V` 모드로 끌어서 맞춥니다.
4. Scene Type을 고르고, 수정했으면 Note에 이유를 적습니다.
5. `Space` 로 저장 후 다음. 라벨을 바꿨으면 프로그램이 자동으로 EDITED로 저장합니다.

하루 마감 루틴 (17:30~18:00)

```
# 1) 전원: 프로그램 종료 후 내 WORK 폴더를 공유폴더로 복사 (예: 공유폴더 /mnt/c/kimchi_share)
cp -r ~/kimchi_labeler/data/work /mnt/c/kimchi_share/work_<이름>

# 2) 문서: 병합 (날짜는 MMDD)
cd ~/kimchi_labeler && source .venv/bin/activate
python scripts/04_merge.py --inputs /mnt/c/kimchi_share/work_A ... /mnt/c/kimchi_share/work_F \
       --out /mnt/c/kimchi_share/work_merged_1014

# 3) QA: 병합본 검사 + RAW 무변경 확인
python scripts/02_validate.py --work /mnt/c/kimchi_share/work_merged_1014
python scripts/00_raw_snapshot.py --verify --work /mnt/c/kimchi_share/work_merged_1014

# 4) 문서: QA Summary (또는 프로그램 메뉴 통계 → QA Summary 생성)
python scripts/05_qa_summary.py --work /mnt/c/kimchi_share/work_merged_1014

# 5) 다음 날 14:00 전원: 병합본으로 내 WORK 교체
rm -rf ~/kimchi_labeler/data/work
cp -r /mnt/c/kimchi_share/work_merged_1014 ~/kimchi_labeler/data/work
```

PM 은 진행현황(통계 → 진행현황)과 병합 충돌 보고서(`merge_conflicts.csv`)를 확인하고 다음 날 목표를 공지합니다. **프로그램 실행 중에는 manifest.csv를 엑셀로 열어 두지 마세요.** 파일이 잠겨 저장이 실패합니다.

자주 나는 문제

증상	해결
[중단] RAW(...) 와 WORK(...) 는 서로 다른 독립 폴더	프로그램 폴더 자체를 RAW 로 고름 → 데이터 폴더 (`~/kimchi_labeler/data/raw`)를 다시 선택
RAW 폴더가 없습니다: .../group2_project/group2_project/...	폴더 선택 창에 상대경로를 입력함 → `/home/...` 전체 경로로 입력하거나 더블클릭으로 이동
먼저 설치를 하세요: bash setup_wsl.sh	`bash setup_wsl.sh` 를 먼저 실행
No module named 'tkinter'	`sudo apt install -y python3-tk` → `rm -rf .venv` → `bash setup_wsl.sh`
ls: cannot access ... kimchi_labeler*.zip	zip 을 Windows **다운로드** 폴더에 받았는지 확인
창 제목줄 한글이 네모(□)	Windows PowerShell 에서 `wsl --shutdown` 후 WSL 다시 열기
externally-managed-environment (pip)	`source .venv/bin/activate` 를 먼저 하고 다시 실행

자료 위치

자료	파일
프로그램 최종본 (배포용)	`kimchi_labeler_final.zip` → `~/kimchi_labeler/`

설치·사용 설명	`kimchi_labeler/README.md`
코드 분석 공부 자료	`kimchi_labeler/docs/STUDY_GUIDE.md`
역할별 코드 요약	`kimchi_labeler/docs/CODE_GUIDE.md`
클릭하면 체크되는 이 가이드	`kimchi_labeler/docs/team/` 의 PDF 4개

최종 산출물과 발표 구성

10/21에는 아래 11개 산출물이 모두 있어야 합니다. 대부분은 스크립트가 자동으로 만들고, 사람은 내용을 확인하고 다듬습니다.

No.	산출물	만드는 방법	담당
☐ 1	라벨링 프로그램	`kimchi_labeler` (`run.sh` / `main.py`)	GUI
☐ 2	소스코드 + Git 이력	Git 저장소 (데이터 제외), 태그 v1.0 · final	PM
☐ 3	FINAL YOLO 라벨	`07_build_final.py` → `data/final/images` , `labels`	PM
☐ 4	Project Baseline	`docs/project_baseline.md`	PM
☐ 5	Class 기준서	`docs/class_guide.md` (+ `configs/classes.yaml`)	문서
☐ 6	BBox 기준서	`docs/bbox_guide.md` (예시 이미지 `docs/images/`)	문서
☐ 7	Dataset Manifest	`10_export_manifest.py` → `manifests/dataset_manifest.csv`	문서
☐ 8	QA Summary	`05_qa_summary.py` → `reports/qa_summary.md`	문서
☐ 9	Test Report	`reports/test_report.md` (Golden · Pilot · Acceptance)	QA
☐ 10	README	`README.md`	GUI·문서
☐ 11	교과 8 Handoff	`08_handoff.py` → `docs/subject08_handoff.md`	문서

발표 구성 (약 10분 기준)

1. 문제와 목표 (PM, 1분): 900장을 믿을 수 있는 정답으로 만드는 것, 100% 가상 데이터임을 명시
2. 데이터 현황 (문서, 1분): Dataset/split, 장면 유형 분포, RAW 사전검사에서 찾은 오류 수
3. 프로그램 구조와 핵심 코드 (GUI·좌표·저장, 3분): GUI와 로직 분리, 원본 좌표 저장, 안전저장, WSL 메뉴 잔상을 직접 만든 메뉴로 해결
4. 시연 (GUI, 2분): 불러오기 → Zoom → BBox 추가 → Class 변경 → 저장 → 재실행 복원 → Validation
5. QA 결과 (QA, 2분): 수정·추가·삭제 수, Class별 BBox, 교차검수와 표본검수 비율, Critical 0
6. 배운 점과 교과 8 연결 (PM, 1분): 한계와 교과 8에서 확인할 주의사항

시연은 네트워크나 PC 문제에 대비해 **10/19 에 2분짜리 화면 녹화본**을 미리 만들어 둡니다.